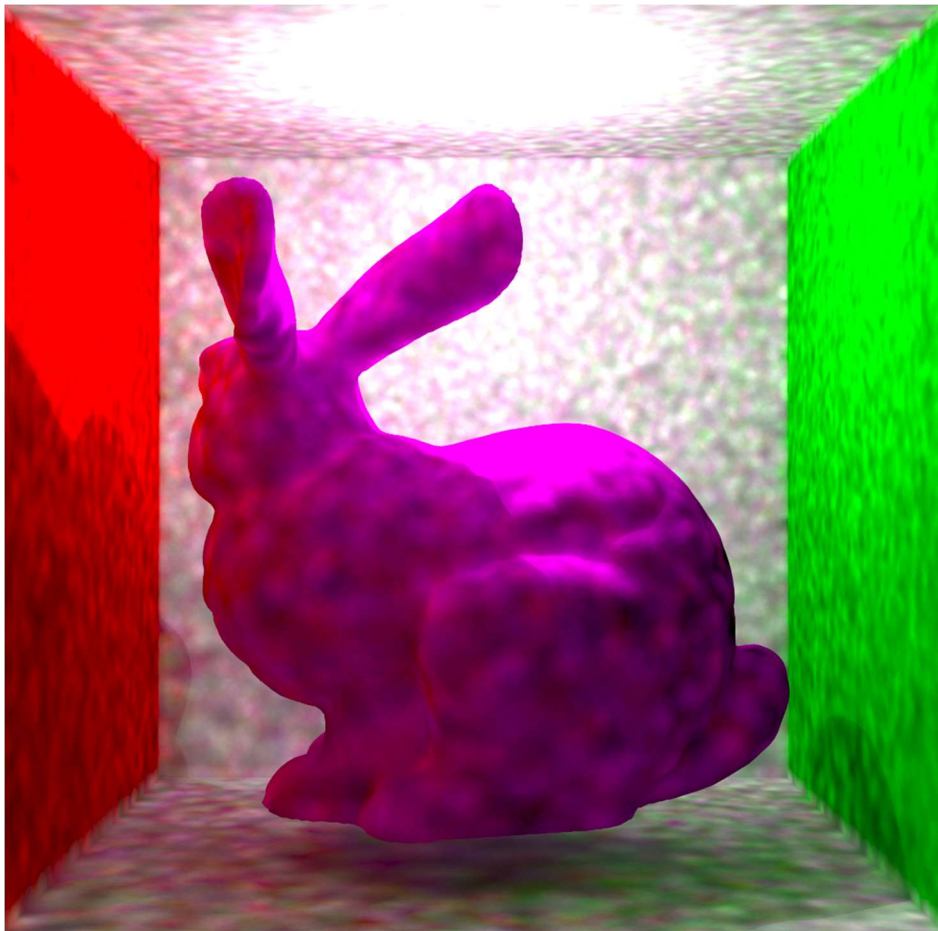


Extensiones

INFORAMTICA GRÁFICA(2025-2026)



Alex Petrov

NIP: 812019 | 10/12/2025

1. Informe extensión significativa (Photon Mapper)

1. **Escribid la ecuación que describe la estimación de radiancia del photon mapping. Para cada término de la ecuación de la estimación de radiancia, explicad brevemente qué significa.**

Podemos estimar la radiancia de la siguiente manera:

$$L_0(x, \vec{\omega}_0) \approx \sum_{p=1}^k f_r(x, \vec{\omega}_p, \vec{\omega}_0) \frac{\Phi_p}{\pi r_k^2}$$

Donde:

- Φ_p es el flujo del fotón
- r_k es la distancia de los fotones más cercanos
- k es el número de fotones más cercanos
- f_r BRDF del material en x , que describe cómo la luz incidente desde la dirección del fotón $\vec{\omega}_p$ se refleja hacia $\vec{\omega}_0$.

2. **Explicad en dos párrafos cómo están relacionadas la Ecuación de Render y la ecuación de estimación de radiancia de la pregunta anterior.**

La Ecuación de Render define la radiancia saliente como una integral continua sobre todas las direcciones de entrada:

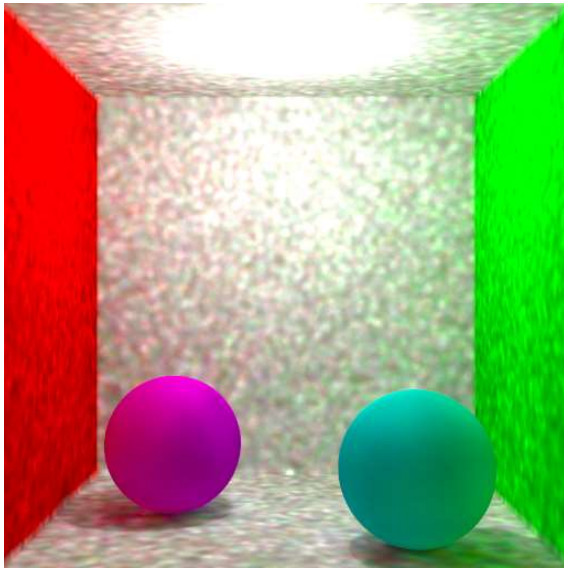
$$L_0(x, \vec{\omega}_0) = L_e(x, \vec{\omega}_0) + \int_{\Omega} L_i(x, \vec{\omega}_i) f_r(x, \vec{\omega}_i, \vec{\omega}_0) |\vec{n} \cdot \vec{\omega}_i| d\vec{\omega}_i$$

En photon mapping, la integral de la ecuación de render se transforma en una suma discreta sobre fotones almacenados. Los fotones representan muestras de $L_i(x, \vec{\omega}_i)$ que han sido transportadas desde las luces a través de la escena. La estimación de radiancia sustituye la integral continua por una suma local de contribuciones dentro de un área finita.

La diferencia clave es que el path tracing evalúa la ecuación de render desde la cámara hacia las luces, mientras que el photon mapping separa el problema en dos fases: primero aproxima la radiancia incidente mediante fotones (desde las luces), y luego evalúa la ecuación de render localmente usando esa aproximación. Esto hace que el photon mapping sea especialmente eficaz para fenómenos difíciles como cáusticas, donde el path tracing converge muy lentamente.

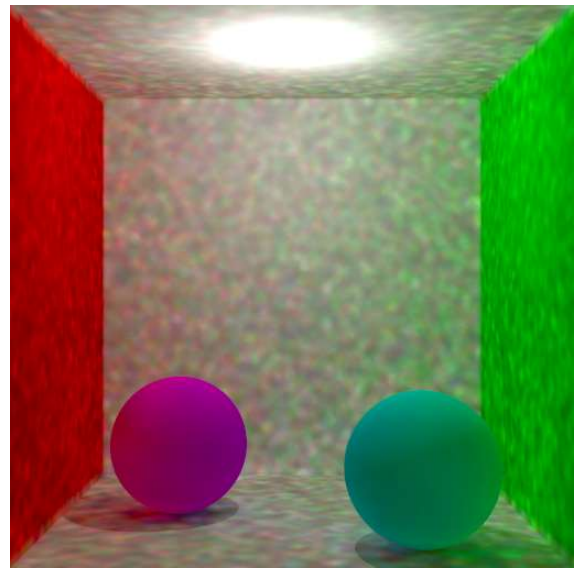
3. **Comparad la iluminación directa obtenida con next-event estimation (rayos de sombra a las luces puntuales) y con los fotones almacenados en el mapa de fotones. Para ello, generad dos renders de la misma escena (una Cornell Box con las dos esferas difusas) con ambas técnicas y poned uno al lado del otro, con la misma técnica de tone mapping con los mismos parámetros.**

Enumerad las diferencias que se pueden observar entre ambas dos imágenes y, para cada una de ellas, explicad brevemente a qué creéis que se debe.



*Imagen de 512x512, 64 muestras/pixel, 100K
photones, 500 vecinos (1m 57.7685s)*

*Almacenando el primer rebote y sin usar next
event estimation*



*Imagen de 512x512, 64 muestras/pixel, 100K
photones, 500 vecinos (1m 46.4885s)*

*No se almacena el primer rebote y se usa next
event estimation*

La imagen obtenida con next-event estimation (NEE) presenta una iluminación directa mucho más limpia y estable que la obtenida mediante photon mapping. Se observa menos ruido en paredes, suelo y techo, así como sombras más definidas bajo las esferas. Esto se debe a que la NEE evalúa explícitamente la contribución de la luz lanzando rayos de sombra hacia la fuente luminosa, reduciendo de forma significativa la varianza en la estimación de la iluminación directa.

En cambio, cuando la iluminación directa se estima a partir de fotones almacenados, aparece más ruido y falta de uniformidad, ya que la radiancia se reconstruye como una densidad local de fotones dentro de un radio finito. Esta aproximación depende fuertemente del número de fotones y del radio de búsqueda, y converge más lentamente.

4. Analizad los efectos de las BSDFs delta en el Photon Mapping. Para ello, renderizad una Cornell Box con dos esferas: una de material plástico (difuso + especular) y otra de material de cristal (transmisivo + especular). ¿Qué efectos de iluminación y apariencia provienen del photon mapping? ¿Qué efectos provienen de la parte de trazado de rayos del algoritmo?

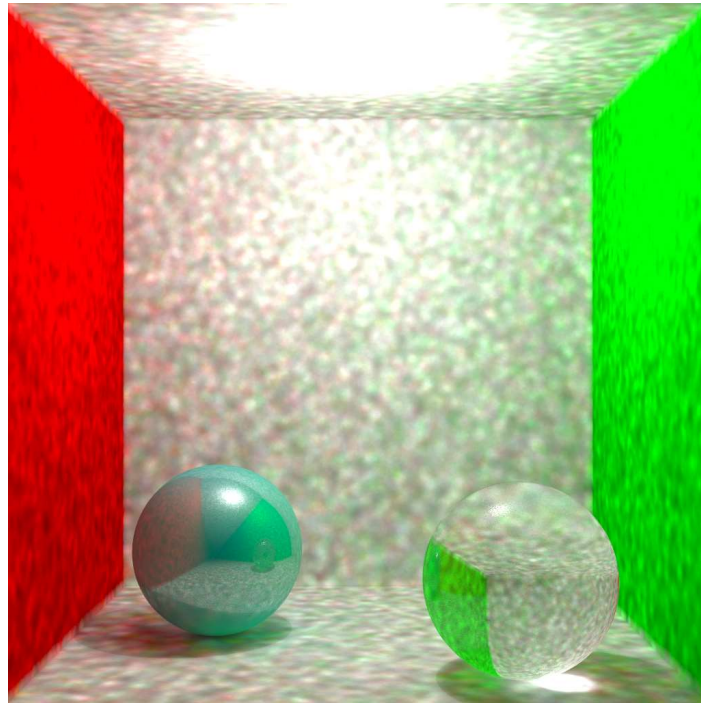


Imagen de 1024x1024, 64 muestras/pixel, 100K fotones, 500 vecinos (5m 26.8043s)

En presencia de BSDFs delta (especular perfecta y transmisión perfecta), el photon mapping contribuye principalmente a la iluminación indirecta difusa y a los efectos que pueden representarse como acumulación de energía sobre superficies no delta. En la escena se observa color bleeding en las paredes y el suelo, así como acumulación de luz bajo la esfera de cristal (patrones de cáustica suavizados si se almacenan fotones cáusticos). Estos efectos provienen del mapa de fotones, que captura cómo la energía luminosa se distribuye globalmente tras múltiples interacciones con superficies difusas.

Por otro lado, los efectos asociados a las interacciones delta provienen exclusivamente de la parte de trazado de rayos del algoritmo. Esto se debe a que las BSDFs delta no pueden estimarse mediante densidad de fotones (no tienen soporte angular), por lo que deben resolverse siguiendo trayectorias deterministas de rayos. En conjunto, el photon mapping aporta la iluminación global y el transporte de energía, mientras que el ray tracing se encarga de la apariencia óptica precisa de los materiales especulares y transmisivos.

5. Analizad y mostrad resultados renderizados utilizando un número creciente de fotones (por ejemplo, $N = \{1K, 10K, 100K\}$) fijando el número máximo de vecinos cercanos en la estimación de radiancia a $k = 10$. Explicad las diferencias entre los resultados en dos párrafos, en términos de coste, calidad de imagen y convergencia.

Cada imagen se renderiza a una resolución de 256x256 píxeles, y 6 rebotes para cada path del algoritmo de path tracing y los fotones. Se usarán 32 muestras por pixel.

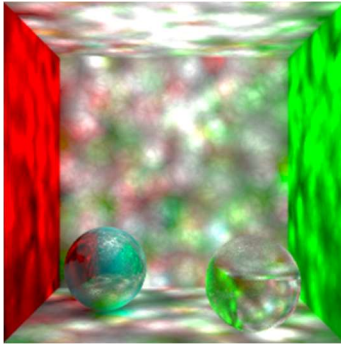


Figura 0.1 – 1K fotones. 10 vecinos

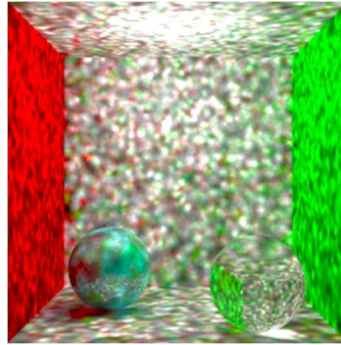


Figura 0.2 - 10K fotones. 10 vecinos

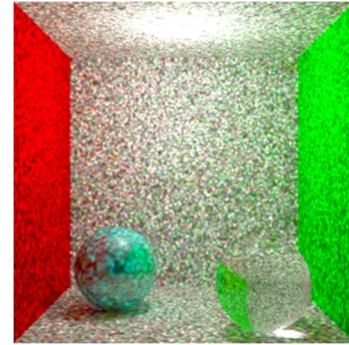


Figura 0.3 - 100K fotones. 10 vecinos

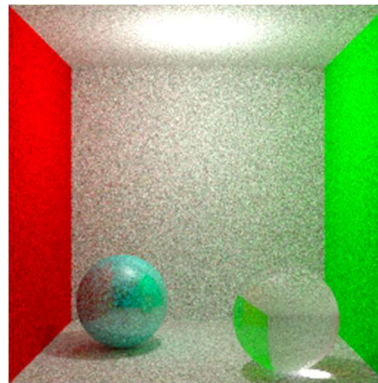


Figura 0.4 - 1M fotones. 10 vecinos

Los tiempos de renderizado han sido los siguientes:

Fotones	Vecinos	Tiempo
1K	10	3.25125s
10K	10	3.58055s
100K	10	6.77039s
1M	10	16.184s

Al aumentar el número de fotones manteniendo fijo el número de vecinos $k=10$, se observa una mejora progresiva en la calidad de imagen. Con 1K fotones la estimación de radiancia es muy ruidosa y aparecen fuertes artefactos de grano, ya que la densidad de fotones es insuficiente para una reconstrucción estable. Con 10K fotones el ruido disminuye notablemente y la iluminación global empieza a ser coherente, mientras que con 100K y 1M fotones la imagen se vuelve mucho más uniforme, con color bleeding y cáusticas mejor definidas. Esto se debe a que, al aumentar la densidad de fotones, la estimación de

radiancia basada en los k vecinos se vuelve más representativa del flujo real de energía.

Desde el punto de vista computacional, el coste de construcción del mapa de fotones crece linealmente con el número de fotones, aumentando tanto el tiempo de trazado como el uso de memoria. Sin embargo, el coste de la estimación de radiancia durante el render permanece aproximadamente constante, ya que el número de vecinos considerados es fijo. En términos de convergencia, el photon mapping converge de forma no sesgada en varianza, pero sesgada en media, es decir, aumentar el número de fotones reduce el ruido (varianza), pero el suavizado impuesto por k introduce un sesgo que no desaparece completamente. Aun así, con suficientes fotones se alcanza una solución visualmente estable y de alta calidad.

6. Ahora, renderizad la misma escena con $N = 100K$ fotones, incrementando el número de vecinos cercanos en la estimación de radiancia (por ejemplo, $k = \{1, 10, 50, 100\}$). Explicad las diferencias entre los resultados en dos párrafos, en términos de coste, calidad de imagen y convergencia.

Cada imagen se renderiza a una resolución de 256×256 píxeles, y 6 rebotes para cada path del algoritmo de path tracing y los fotones. Se usarán 32 muestras por pixel.

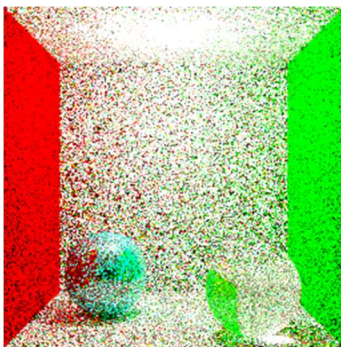


Figura 0.5 - 100K fotones. 1 vecino

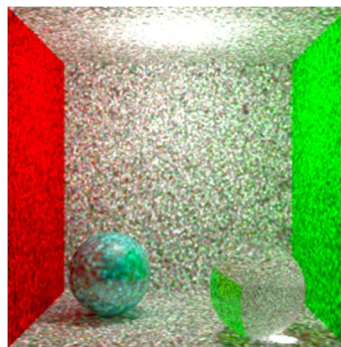


Figura 0.6 - 100K fotones. 10 vecinos

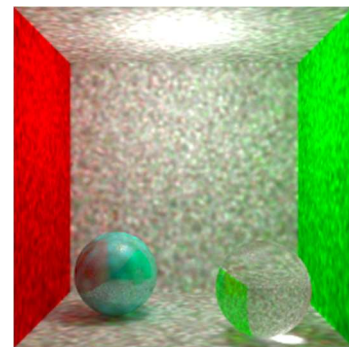


Figura 0.7 - 100K fotones. 50 vecinos

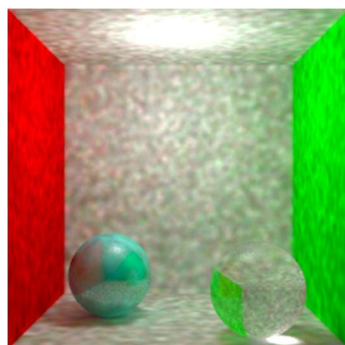


Figura 0.8 - 100K fotones. 100 vecinos

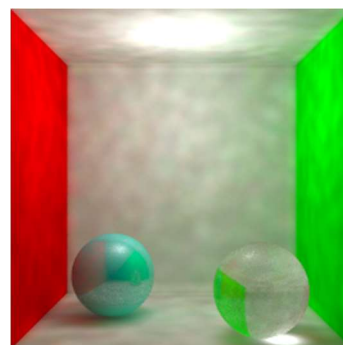


Figura 0.9 - 100K fotones. 500 vecinos

Los tiempos de renderizado han sido los siguientes:

Fotones	Vecinos	Tiempo
100K	1	2.71083s
100K	50	13.6814s
100K	100	30.3112s
100K	500	2m 6.61105s

Al aumentar el número de vecinos k manteniendo fijo el número de fotones en 100K, se observa una clara transición entre ruido y suavizado. Con $k = 1$ la imagen es extremadamente ruidosa, ya que la estimación de radiancia depende de un único fotón y presenta una varianza muy alta. Al pasar a $k = 10$ y $k = 50$, el ruido disminuye de forma notable y la iluminación global se vuelve más coherente, manteniendo todavía detalles como el color bleeding y las sombras suaves. Con valores altos como $k = 100$ o 500 , la imagen se vuelve muy suave, pero empiezan a perderse detalles finos y contraste, especialmente en cáusticas y zonas de contacto, debido al exceso de promediado espacial.

Desde el punto de vista computacional, el coste de la estimación de radiancia crece significativamente con k , como reflejan los tiempos de render: mientras que k bajos son rápidos, valores altos incrementan mucho el tiempo al requerir búsquedas más costosas de vecinos en el mapa de fotones. En términos de convergencia, aumentar k reduce la varianza, pero incrementa el sesgo del estimador: la solución converge más rápido a una imagen suave, pero no necesariamente más precisa físicamente. Por ello, existe un compromiso entre calidad visual, fidelidad y coste, siendo valores intermedios de k los que suelen ofrecer el mejor equilibrio práctico.

7. Explicad en dos párrafos cómo extenderíais vuestro photon mapper para tener en cuenta medios participativos.

Para extender el photon mapping a medios participativos (como niebla, humo o líquidos), sería necesario permitir que los fotones interactúen no solo con superficies, sino también con el volumen del medio. Durante la fase de emisión de fotones, cada fotón se propagaría por el volumen y se muestrearía una distancia de colisión usando el coeficiente de extinción del medio. En cada interacción volumétrica, el fotón podría ser absorbido o dispersado según una función de fase, y los eventos de dispersión se almacenarían en un mapa de fotones volumétrico.

En la fase de render, la estimación de radiancia se extendería al volumen integrando la contribución de los fotones cercanos a lo largo del rayo de cámara. Esto implica realizar una estimación de densidad volumétrica en lugar de superficial, teniendo en cuenta la distancia recorrida en el medio y la función de fase. De este modo, el photon mapping permitiría capturar efectos como god rays, dispersión múltiple y atenuación de la luz, combinándose con el trazado de rayos para manejar superficies y eventos delta.

8. Explicad en dos párrafos cómo extenderíais vuestro photon mapper para render transitorio, teniendo en cuenta la luz en movimiento.

Para soportar render transitorio, el photon mapper debería incorporar el tiempo como una dimensión adicional. Cada fotón almacenado llevaría asociado un tiempo de llegada, calculado a partir de la distancia recorrida desde la fuente

de luz y su velocidad de propagación. En escenas con luz en movimiento, la posición y orientación de la fuente variarían en el tiempo, por lo que la emisión de fotones debería realizarse sobre múltiples instantes temporales, generando un mapa de fotones dependiente del tiempo o varios mapas discretos por intervalo.

Durante el render, la radiancia se estimaría no solo en función del espacio, sino también del instante de observación, acumulando únicamente los fotones cuya marca temporal coincida con la ventana temporal del sensor. Esto permitiría simular fenómenos como frentes de onda luminosos, retardos de iluminación y efectos de time-of-flight. Aunque el coste computacional y de memoria aumenta notablemente, esta extensión hace posible renderizar iluminación dinámica de forma físicamente consistente.

2. Informe extensión menor (mallas de triángulos)

1. Crea una escena, una Cornell Box, por ejemplo, con un objeto en el que cada triángulo tenga una única normal (usando las normales de las caras de los triángulos). Vuelve a generar esa misma escena pero esta vez aplicando suavizado de normales. ¿Qué cambios en la apariencia del objeto se pueden apreciar?

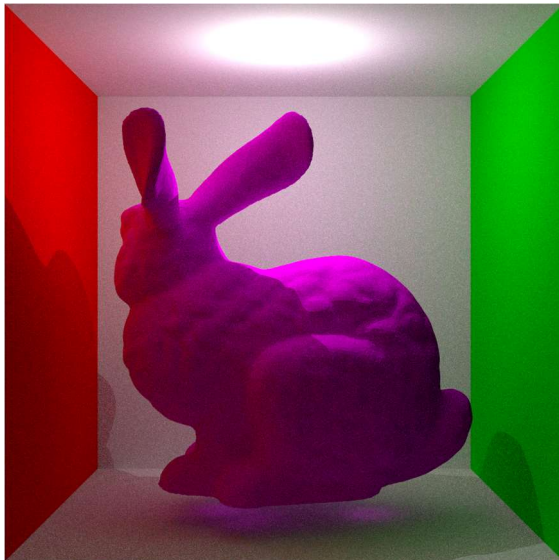


Imagen de 1024x1024, 64 muestras/pixel sin suavizado

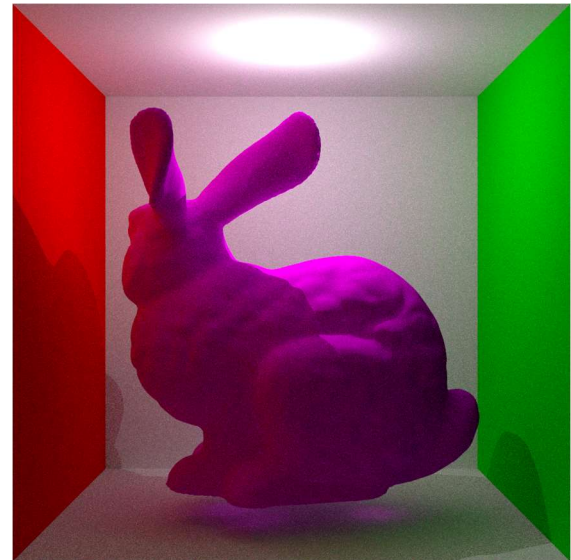


Imagen de 1024x1024, 64 muestras/pixel con suavizado

En la render sin suavizado, el conejo muestra un aspecto más "poligonal", se aprecian cambios bruscos de iluminación entre triángulos, sobre todo en zonas de curvatura suave (orejas, lomo y barriga). Esto ocurre porque cada triángulo usa una única normal constante (la de su cara), así que el término difuso (y cualquier componente especular si la hubiese) cambia a saltos al cruzar aristas. El resultado es "facetas" visibles y transiciones discontinuas en el sombreado, aunque la silueta geométrica sea la misma.

En la render con suavizado, la iluminación varía de forma continua, las transiciones de luz y sombra sobre el cuerpo del conejo son mucho más suaves y desaparece gran parte del facetado. Esto se debe a que la normal en el punto de intersección se interpola a partir de las normales por vértice (obtenidas promediando normales de caras adyacentes) usando coordenadas baricéntricas, y luego se normaliza. La geometría no cambia (la silueta sigue igual), pero el campo de normales es más "suave", por eso el objeto parece más redondeado y con menos "caras" visibles.

2. Realiza un análisis del coste temporal de renderizado mostrando una comparativa del tiempo entre la implementación base del path tracer y usando estructuras de aceleración (BVHs) para el cálculo de las intersecciones. Para ello, renderiza una escena del mismo objeto variando el número de triángulos de la malla.

En la implementación base del path tracer, el cálculo de intersecciones rayo-triángulo se realiza de forma lineal (se prueban todos los triángulos). En la práctica, incluso para una escena sencilla como la Cornell Box con una única instancia del Stanford Bunny, el tiempo de renderizado sin BVH crece rápidamente hasta ser inviable cuando se incrementa el número de triángulos. En las pruebas realizadas, los renders sin BVH para recuentos altos de triángulos no pudieron completarse en un tiempo razonable y se abortaron.

Con BVH, el tiempo de renderizado es mucho menor y sí permite completar las mismas escenas con un número creciente de triángulos. La razón es que el BVH permite descartar grandes grupos de primitivas mediante intersecciones rayo-AABB, reduciendo drásticamente el número de tests rayo-triángulo evaluados por rayo. Como resultado, mientras que sin BVH el coste aumenta aproximadamente de forma proporcional al número de triángulos (y se vuelve impracticable), con BVH el crecimiento es mucho más contenido y el render sigue siendo realizable para geometría compleja.

3. Asumiendo que para una malla de triángulos texturizada tenemos el mapa UV para los vértices de los triángulos, es decir, sus coordenadas UV (2D) en la textura. Escribe las ecuaciones que necesitarías para obtener las coordenadas UV de un punto en 3D sabiendo sus coordenadas baricéntricas (b_0, b_1, b_2) de intersección en un triángulo, explicando cada uno de los términos.

Si en el triángulo tiene las UV de sus vértices $uv_0 = (u_0, v_0)$, $uv_1 = (u_1, v_1)$, $uv_2 = (u_2, v_2)$ y para el punto de intersección se conocen sus coordenadas baricéntricas (b_0, b_1, b_2) , $b_0 + b_1 + b_2 = 1$, las UV del punto se obtienen por interpolación baricéntrica:

$$uv(P) = b_0 uv_0 + b_1 uv_1 + b_2 uv_2$$

Donde:

- uv_0, uv_1, uv_2 coordenadas UV (2D) asociadas a cada vértice del triángulo en el espacio de la textura.
- b_0, b_1, b_2 pesos baricéntricos del punto de intersección dentro del triángulo; indican "cuánto" contribuye cada vértice al punto.
- $uv(P)$ coordenadas UV interpoladas para el punto 3D P , que luego se usan para muestrear la textura.

4. Supon que se quisiera expandir el path tracer para soportar bump mapping, estimando las normales de la superficie a partir de un mapa de altura. Describe paso a paso la implementación que habría que realizar para lograrlo. No es necesario implementar nada, simplemente describir los pasos a seguir.

Para soportar bump mapping en el path tracer, el primer paso es disponer de coordenadas UV en el punto de intersección, que se obtienen interpolando las UV de los vértices del triángulo mediante coordenadas baricéntricas. A continuación, el material debe poder acceder a un mapa de alturas, del que

se obtiene un valor de altura para las UV del punto. Con este mapa se estiman las derivadas de la altura en las direcciones u y v usando diferencias finitas.

Después, se construye un sistema de referencia local en el punto de intersección (normal, tangente y bitangente). El gradiente de la altura se usa para perturbar la normal en el espacio tangente, y esta normal perturbada se transforma al espacio del mundo mediante la base tangente-bitangente-normal. Finalmente, la normal resultante se normaliza y se utiliza únicamente en el cálculo de la iluminación (evaluación de la BSDF), sin modificar la geometría ni el cálculo de intersecciones. De este modo, el bump mapping simula relieve en la superficie alterando solo el sombreado.